

模型：微分方程和仿真

Contents

- 11.1. 马尔萨斯模型
- 11.2. Logistic模型
- 11.3. 微分方程建模方法
- 11.4. 低速风洞非定向响应建模

11.1. 马尔萨斯模型

马尔萨斯模型是人口增长模型中最简单的模型，它由英国牧师马尔萨斯在1798年提出。他利用在教堂工作的机会，收集英国100多年的人口数据，发现人口的相对增长率是常数。在这个基础上，建立了一个描述人口增长的模型，也就是著名的“马尔萨斯人口模型”。

在这个模型中，最重要的概念是相对增长率。

$$\frac{\dot{u}}{u} = r$$

这里就涉及到微分的概念，变量 u 对时间 t 的导数，也就是变化率(这里可以称之为增长率)，记作 $\dot{u}$ 。

$$\dot{u} = \frac{du}{dt}$$

那么“马尔萨斯人口模型”表达为微分方程是这样的形式。

$$\frac{du/dt}{u} = \alpha \rightarrow \frac{du}{dt} = \alpha u$$

其中，时间为 t ，人口 u 为依赖于时间的函数，相对增长率是 α ($\alpha > 0$)。

这个方程的解很容易通过不定积分求解。

$$\int \frac{du}{u} = \int \alpha dt \rightarrow \ln u = \alpha t + C \rightarrow u(t) = e^{\alpha t + C} = ne^{\alpha t}$$

这个解是一个指数函数！众所周知，指数函数的增长是非常快的。这在一定的程度上导致了社会主义国家考虑对人口增长进行控制。

下面我们用 `sympy` 来求解这个常微分方程。

```
import sympy as sp
from IPython.display import display

# 定义符号
t = sp.Symbol('t')
u = sp.Function('u')
alpha = sp.Symbol('alpha', real=True, positive=True)

# 定义微分方程
# sp.Eq(lhs, rhs) 创建一个等式
# u(t).diff(t) 是 u 对 t 的一阶导数
ode = sp.Eq(u(t).diff(t), alpha * u(t))

print("微分方程:")
display(ode)

# 求解微分方程
# dsolve 是 sympy 中用于求解常微分方程的函数
solution = sp.dsolve(ode)

print("\n通解:")
display(solution)

# 假设初始条件 u(0) = u_0
u0 = sp.Symbol('u0', positive=True)
# 求解包含初始条件的特解
# ics = {u(t0): u_t0}
particular_solution = sp.dsolve(ode, ics={u(0): u0})

print("\n给定初始条件 u(0)=u0 的特解:")
display(particular_solution)
```

微分方程：

$$\frac{d}{dt}u(t) = \alpha u(t)$$

通解：

$$u(t) = C_1 e^{\alpha t}$$

给定初始条件 $u(0)=u_0$ 的特解：

$$u(t) = u_0 e^{\alpha t}$$

11.1.1. 与UN预测数据对比

我们可以从[UN官网](#)下载人口预测数据。数据的格式是csv，第二表格包含了世界各国从2025年到2100年的人口预测数据。

我们通过 `requests` 下载数据，并用 `pandas` 读取数据。注意实际的数据位置、文件名可能会发生改变，需要根据实际情况修改。

然后我们来看看，马尔萨斯模型数据与UN预测数据的差异。



```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import requests
import io
from scipy.optimize import curve_fit

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# --- 1. 下载和处理数据 ---
# UN WPP 2022 数据集的URL (注意: 这个URL可能会变动)
url = 'https://population.un.org/wpp/Download/Files/1_Indicators%20(Standard)/CSV_F

print(f"正在从 {url} 下载数据...")
try:
    response = requests.get(url, timeout=20)
    response.raise_for_status() # 如果下载失败则抛出异常

    # 使用 pandas 读取数据
    # 使用 io.StringIO 将下载的文本内容模拟成一个文件
    csv_content = io.StringIO(response.text)
    df = pd.read_csv(csv_content)

    # 筛选世界总人口数据
    world_pop = df[(df['Location'] == 'World') & (df['Variant'] == 'Medium')]

    # 提取年份和人口数据 (人口单位是千人)
    years = world_pop['Time'].values
    population_thousands = world_pop['PopTotal'].values

    # 转换为实际人口
    population = population_thousands * 1000

    print("数据下载和处理成功。")

    # --- 2. 拟合马尔萨斯模型 ---
    # 定义马尔萨斯模型函数
    def malthus_model(t, u0, alpha):
        return u0 * np.exp(alpha * (t - years[0]))

    # 使用 curve_fit 进行拟合
    # p0 是初始参数猜测
    popt, pcov = curve_fit(malthus_model, years, population, p0=(population[0], 0.0

    u0_fit, alpha_fit = popt
    print(f"\n拟合得到的参数: u0 = {u0_fit:e}, alpha = {alpha_fit:.4f}")

    # --- 3. 绘图比较 ---
    plt.figure(figsize=(10, 6))

    # 绘制UN预测数据
    plt.plot(years, population, 'ko', label='UN 预测数据 (WPP 2022)')

    # 绘制拟合的马尔萨斯模型曲线
    t_plot = np.linspace(years[0], years[-1], 200)
    pop_fit = malthus_model(t_plot, u0_fit, alpha_fit)

```

```
plt.plot(t_plot, pop_fit, 'r-', label=f'拟合的马尔萨斯模型 ( $\alpha={\alpha\_fit:.4f}$ )')

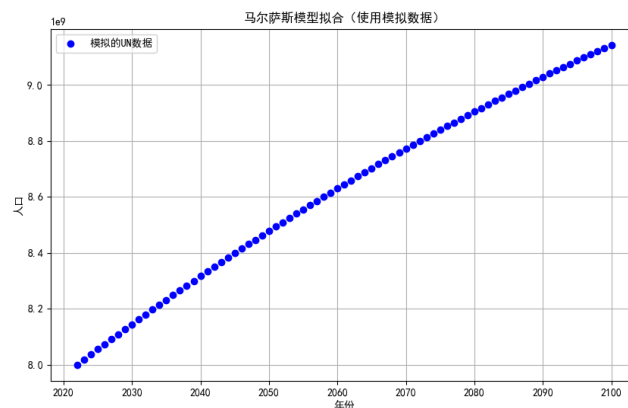
plt.title('马尔萨斯模型与联合国世界人口预测数据对比')
plt.xlabel('年份')
plt.ylabel('人口')
plt.legend()
plt.grid(True)
plt.ticklabel_format(style='sci', axis='y', scilimits=(9,9)) # 使用科学计数法显示
plt.show()

except requests.exceptions.RequestException as e:
    print(f"数据下载失败: {e}")
    print("将使用示例数据进行演示。")
    # 如果下载失败, 使用一组示例数据
    years = np.arange(2022, 2101)
    # 模拟一个增长放缓的人口曲线
    p_start = 8e9
    p_peak = 10.4e9
    growth_rate = 0.01
    population = p_peak / (1 + (p_peak/p_start - 1) * np.exp(-growth_rate * (years

plt.figure(figsize=(10, 6))
plt.plot(years, population, 'bo', label='模拟的UN数据')
plt.title('马尔萨斯模型拟合 (使用模拟数据)')
plt.xlabel('年份')
plt.ylabel('人口')
plt.legend()
plt.grid(True)
plt.show()
```

正在从 [https://population.un.org/wpp/Download/Files/1_Indicators%20\(Standard\)/CSV_FI](https://population.un.org/wpp/Download/Files/1_Indicators%20(Standard)/CSV_FI)

数据下载失败: 404 Client Error: The requested content does not exist. for url: https
将使用示例数据进行演示。



../_images/30de2e1c2f74488ce384880b6980ec0216776ce9729ca269a97fe70f197141b9.png

从图上看，马尔萨斯模型预测的人口增长（红色曲线）与联合国的预测（黑点）在早期比较吻合，但随着时间推移，差异越来越大。马尔萨斯模型预测人口将无限增长，而联合国的预测显示人口增长将放缓，并在本世纪末达到一个峰值。

本质上来讲，马尔萨斯模型仅仅考虑人口相对增长率的线性特征，没有考虑非线性的饱和特征。也就是，在人口较少时，人口的增长所受的限制很少，能够出现指数增长；而当人口达到一定的数量是，生存环境、生态资源、社会因素等都会对人口增长产生限制，导致人口增长率逐渐减小，最终趋近于0。

这反映了微分方程模型的适用范围问题。在其适用条件和假设成立的范围内，模型能够准确描述系统的动态行为；但当系统偏离这些基本假设时，模型的预测能力将显著降低。这是数学建模中普遍存在的局限性。

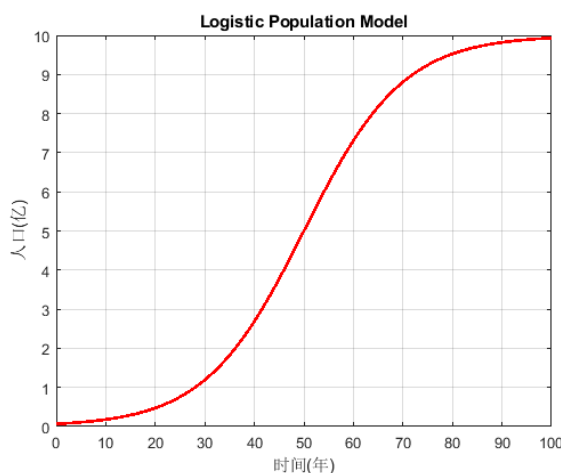
基于以上分析，我们需要构建一个能够刻画人口增长非线性特征的数学模型，以更准确地描述人口动态变化规律。这就引出了下面将要介绍的Logistic模型。

11.2. Logistic模型

Logistic模型为什么叫做Logistic模型呢？因为它的解是一个Logistic函数。什么叫Logistic函数呢？它的形式是这样的：

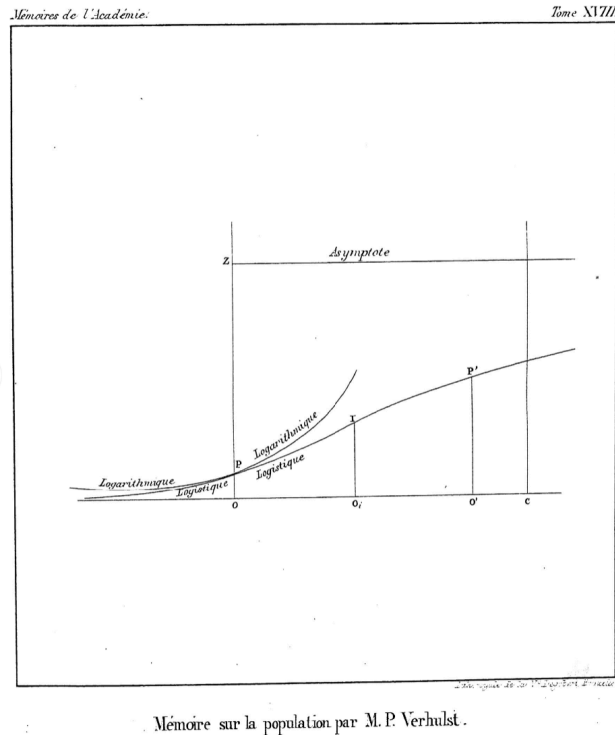
$$f(x) = \frac{L}{1 + e^{-k(x-x_0)}}$$

这里 L 是函数的最大值， k 是增长率， x_0 是函数的中点。



Logistic函数

至于这个函数为什么叫做Logistic函数呢？因为法国数学家 Pierre Franois Verhulst就是这么命名的，他在1845年的[论文](#)提出了这个函数来描述人口增长。



Verhulst论文截图

Verhulst在论文里面写到：*Nous donnerons le nom de logistique à la courbe...* 意思就是我们给这个曲线命名为Logistic曲线。说到底就是他给这个函数起了个名字。

11.2.1. Logistic模型的推导

Logistic模型的基本思想是，人口的相对增长率不再是一个常数，而是随着人口数量 u 的增加而减小。最简单的减小关系是线性关系：

$$\frac{\dot{u}}{u} = r(u) = \alpha - \beta u$$

其中 α 是初始的相对增长率， β 是一个表示环境阻力的系数。当 u 很小时， $r(u) \approx \alpha$ ，模型近似于马尔萨斯模型。当 u 增大时， $r(u)$ 减小。

将上式变形，得到Logistic微分方程：

$$\frac{du}{dt} = u(\alpha - \beta u) = \alpha u \left(1 - \frac{\beta}{\alpha} u\right)$$

我们通常令 $K = \frac{\alpha}{\beta}$ ，称之为环境容量（Carrying Capacity）。这样方程就写成了更常见的形式：

$$\frac{du}{dt} = \alpha u \left(1 - \frac{u}{K}\right)$$

这个方程的解就是前面提到的Logistic函数。下面我们同样用 `sympy` 求解。

```
import sympy as sp
from IPython.display import display

# 定义符号
t = sp.Symbol('t')
u = sp.Function('u')
alpha = sp.Symbol('alpha', real=True, positive=True)
K = sp.Symbol('K', real=True, positive=True) # 环境容量

# 定义Logistic微分方程
logistic_ode = sp.Eq(u(t).diff(t), alpha * u(t) * (1 - u(t)/K))

print("Logistic 微分方程:")
display(logistic_ode)

# 求解微分方程
solution = sp.dsolve(logistic_ode)

print("\n通解:")
display(solution)

# 假设初始条件 u(0) = u0
u0 = sp.Symbol('u0', positive=True)
particular_solution = sp.dsolve(logistic_ode, ics={u(0): u0})

print("\n给定初始条件 u(0)=u0 的特解:")
# simplify() 可以让表达式更美观
display(sp.simplify(particular_solution))
```

Logistic 微分方程：

$$\frac{d}{dt}u(t) = \alpha \left(1 - \frac{u(t)}{K}\right)u(t)$$

通解：

$$u(t) = \frac{Ke^{C_1K+\alpha t}}{e^{C_1K+\alpha t} - 1}$$

给定初始条件 $u(0)=u_0$ 的特解：

$$u(t) = \frac{Ku_0e^{\alpha t}}{K + u_0e^{\alpha t} - u_0}$$

11.2.2. 传染病问题模拟

Logistic模型的一个经典应用是模拟传染病的传播，例如SIR模型（易感者-感染者-康复者）。一个简化的场景是，在一个固定总人口中，感染者的增长。

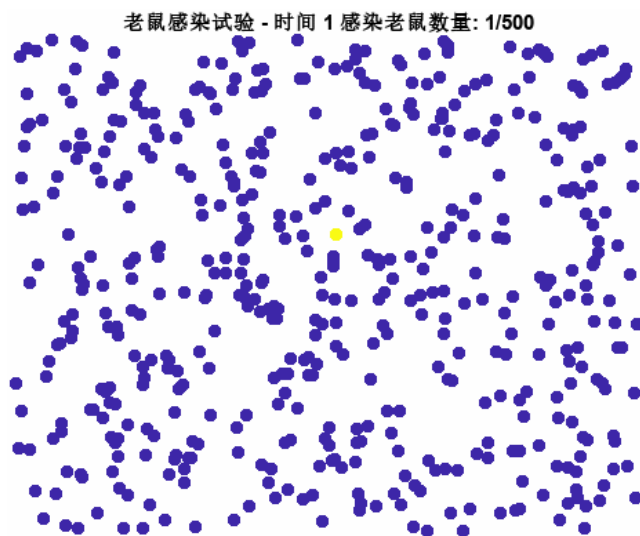
假设总人口为 M ，已感染人数为 $u(t)$ ，健康但易感的人数为 $v(t)$ ，则 $u + v = M$ 。感染率被假设为正比于已感染人数和健康人数的乘积（因为需要两者接触才会发生感染），比例系数为 β 。

$$\frac{du}{dt} = \beta uv = \beta u(M - u)$$

这正是Logistic方程。

原文中有一个有趣的二维空间老鼠传染模拟，虽然和主题关系不大，但我们可以用

`matplotlib` 的动画功能来复现这个过程。



老鼠感染过程

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

# --- 1. 初始化参数 ---
grid_size = 50 # 网格大小
n_mice = 200 # 老鼠总数
infection_prob = 0.1 # 接触时的感染概率
infection_radius = 2 # 感染半径

# --- 2. 初始化老鼠 ---
# 随机放置老鼠
positions = np.random.rand(n_mice, 2) * grid_size
# 0: 健康, 1: 感染
status = np.zeros(n_mice, dtype=int)
# 随机选择一只初始病鼠
initial_infected = np.random.randint(0, n_mice)
status[initial_infected] = 1

# --- 3. 设置图表 ---
fig, ax = plt.subplots(figsize=(7, 7))
ax.set_xlim(0, grid_size)
ax.set_ylim(0, grid_size)
ax.set_xticks([])
ax.set_yticks([])
ax.set_title("老鼠传染病传播模拟")

# 创建散点图对象, 后续只更新其数据
healthy_scatter = ax.scatter([], [], c='blue', label='健康')
infected_scatter = ax.scatter([], [], c='red', label='感染')
ax.legend()

# --- 4. 定义动画更新函数 ---
def update(frame):
    global positions, status

    # a. 老鼠随机移动
    # 在当前位置上增加一个小的随机位移
    positions += np.random.randn(n_mice, 2) * 0.5
    # 处理边界, 让老鼠“反弹”
    positions = np.clip(positions, 0, grid_size)

    # b. 处理感染过程
    infected_indices = np.where(status == 1)[0]
    healthy_indices = np.where(status == 0)[0]

    # 如果没有病鼠或没有健康鼠了, 就停止
    if len(infected_indices) == 0 or len(healthy_indices) == 0:
        ani.event_source.stop() # 停止动画
        return healthy_scatter, infected_scatter

    # 遍历所有病鼠
    for i_idx in infected_indices:
        # 计算该病鼠与其他所有健康鼠的距离
        distances = np.linalg.norm(positions[healthy_indices] - positions[i_idx], 2)

        # 找到在感染半径内的健康鼠

```


11.3. 微分方程建模方法

从马尔萨斯模型到Logistic模型，可以看到利用微分的概念求解实际问题的一般过程：

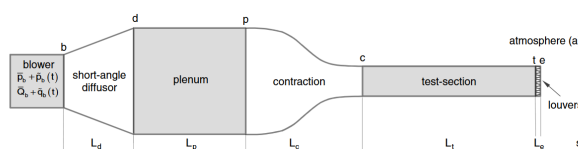
1. 确定考察变量（人口、染病老鼠）；
2. 考察变量的变化规律（变化率）；
3. 列写微分方程
4. 分析初始条件、边界条件和求解条件
5. 讨论方程的解
6. 刻画解的变化规律和特征
7. 讨论解的适用条件

对于简单的微分方程，可以通过符号积分的方式得到解析解。对于更加复杂的微分、代数方程，则需要使用数值方法求解，例如 `scipy.integrate.solve_ivp`，这将在后续的笔记中介绍。

11.4. 低速风洞非定向响应建模

下面用一个例子来说明这个方法在风洞设计模型中的应用。

在低速风洞设计中，非定向响应是一个重要的性能指标。非定向响应描述了风洞在不同来流方向下的流场均匀性和稳定性。为了建立非定向响应的数学模型，我们可以采用微分方程的方法。



低速风洞简化模型

将一个直流低速风洞简化为如图的拟一维模型，风洞的动量方程可以写为：

$$\frac{\partial u}{\partial t} ds + \frac{1}{2}(1 + k_l) d(u^2) + \frac{dp}{\rho} = 0.$$

为了简化分析，引入来流平均量与脉动量的分解：

$$p_b(t) = \bar{p}_b + \tilde{p}_b(t), \quad Q_b(t) = \bar{Q}_b + \tilde{q}_b(t).$$

这些平均量 $\bar{p}_b$ 与 $\bar{Q}_b$ 可按周期平均定义：

$$\bar{Q}_b = \frac{1}{T_p} \int_0^{T_p} Q_b(t) dt, \quad \bar{p}_b = \frac{1}{T_p} \int_0^{T_p} p_b(t) dt.$$

经过推导可以得到脉动量的微分方程：

$$\begin{aligned} & \frac{A_\infty}{(1+k_l)U_\infty} \\ & \left[\int_0^{t'} \frac{1}{A_c(s)} ds + \frac{L_e}{A_e(t)} + L_t \tilde{\varepsilon}'_\infty(t) + \left(\frac{A_\infty}{A_e(t)} \right)^2 \tilde{\varepsilon}_\infty(t) \right] \\ & = \left(\frac{A_\infty}{\bar{A}_e} \right)^2 \frac{\tilde{a}_e(t)}{\bar{A}_e} + \frac{A_\infty L_e}{(1+k_l)\bar{A}_e^2 U_\infty} \tilde{a}'_e(t) + \frac{\tilde{p}_b(t)}{(1+k_l)\rho U_\infty^2}. \end{aligned}$$

根据小量假设， $\tilde{a}_e(t) \ll \bar{A}_e$ ， $\tilde{\varepsilon}_\infty(t) \ll U_\infty$ ，可以忽略高阶小量项，得到线性化的脉动量微分方程。

进一步将扰动速度无量纲化为 $\varepsilon_\infty(t) = \tilde{\varepsilon}_\infty(t)/U_\infty$ ，线性系统可写成：

$$\tau \varepsilon'_\infty(t) + \varepsilon_\infty(t) = g(t).$$

其中风洞的时间常数定义为：

$$\tau = \left(\frac{\bar{A}_e}{A_\infty} \right)^2 \frac{L_t}{U_\infty}.$$

相应的截止频率为：

$$f_c = \frac{2\pi}{\tau} = 2\pi \left(\frac{A_\infty}{\bar{A}_e} \right)^2 \frac{U_\infty}{L_t}.$$

无量纲激励函数可表达为：

$$g(t) = \frac{\tilde{a}_e(t)}{\bar{A}_e} + \frac{\bar{A}_e L_e}{A_\infty U_\infty} \frac{\tilde{a}'_e(t)}{\bar{A}_e} + \left(\frac{\bar{A}_e}{A_\infty} \right)^2 \frac{\tilde{p}_b(t)}{\rho U_\infty^2}.$$

上述一阶线性方程刻画了风洞非定向响应对入口面积扰动与回流压力扰动的动态敏感性，为后续的控制策略设计提供了基础。

当然，对于风洞的声学响应，也可以采用类似的方法进行建模和分析。这在越来越重视风洞动态试验、低空经济飞行等领域具有重要意义。